AI Skills Roadmap for a PostgreSQL DBA

pgvector, Vector Databases and RAG: 10-week free learning plan with day-by-day practice

Prepared for Kiran | Start date: Day 1 = 8 October 2026

1. Why this skill, and the big picture

Vector search (pgvector) is the most natural AI skill for a PostgreSQL DBA. It builds on what you already know, and companies running AI features on Postgres need people who can index, tune, back up and monitor vector workloads in production.

Roadmap overview (8 to 10 weeks)

Weeks	Focus	What you learn
1-2	AI/ML basics	Embeddings, LLMs, RAG, vector database concepts
3-4	Python essentials	psycopg, pandas, calling APIs, sentence-transformers
5-6	pgvector hands-on	Store embeddings, similarity search, IVFFlat vs HNSW tuning
7-8	RAG project	Chunk, embed, store, hybrid search, answer with a local LLM
9-10	DBA production topics	Index memory, vacuum, backup, replication, monitoring, scaling

Free resources

- Generative AI for Everyone (DeepLearning.AI on Coursera, audit for free)
- Intro to Large Language Models (Andrej Karpathy, YouTube)
- Vector Databases: from Embeddings to Applications (DeepLearning.AI short course)
- Building Applications with Vector Databases (DeepLearning.AI short course)
- Python for Everybody (freeCodeCamp / Coursera audit)
- Hugging Face free NLP / LLM course
- Microsoft Generative AI for Beginners (GitHub)
- pgvector GitHub README, plus free Supabase and Neon pgvector guides
- Ollama for running free local LLMs on your own machine

How to use this plan

- Plan for 1.5 to 2 hours every day. If you miss a day, continue from where you stopped; do not skip tasks.
- Tick the Done box each day. Every Sunday (day 7, 14, ...) is a review and writing day.
- Keep every script in one GitHub repo (for example pgvector-lab) and commit daily.
- Write what you learn as blog posts on your Sphinx site. This is your proof of skill for employers.
- Use a notes file for every error you hit and how you fixed it. Production-style troubleshooting is the real skill.

Portfolio deliverables (by day 56)

- GitHub repo with scripts and a clear README
- 3 blog posts: pgvector indexes compared, recall vs latency tuning, running RAG on PostgreSQL
- A short architecture diagram of your RAG setup
- Updated resume and LinkedIn mentioning pgvector, RAG and vector search tuning

2. Day-by-day plan

Week 1: AI basics (Days 1-7)

Goal: Understand what LLMs, embeddings and RAG are.

Day	Task (about 1.5 to 2 hours)	Done
1	SETUP DAY: Install Docker, Python 3.11+, Git. Create a venv and a GitHub repo called pgvector-lab. Start Generative AI for Everyone, week 1.	[ ]
2	Finish Generative AI for Everyone week 1. Write 5 lines: what is an LLM, what are tokens, what is a prompt.	[ ]
3	Start the DeepLearning.AI course Vector Databases: from Embeddings to Applications. Lessons 1-2: what an embedding is.	[ ]
4	Watch Karpathy's Intro to Large Language Models (about 1 hour). Take notes on how LLMs are trained and used.	[ ]
5	Generative AI for Everyone week 2. Write RAG in your own words (retrieve, augment, generate).	[ ]
6	Install Ollama and run a small local model (for example llama3.2 or mistral). Ask it 10 PostgreSQL questions and note wrong answers.	[ ]
7	REVIEW: Write a 1-page summary, AI concepts for a DBA. This becomes your first blog draft.	[ ]

Week 2: Vector database concepts (Days 8-14)

Goal: Learn how similarity search works before touching pgvector.

Day	Task (about 1.5 to 2 hours)	Done
8	Continue the Vector Databases course, lessons 3-4: similarity metrics.	[ ]
9	Compare cosine, L2 and inner product with a small NumPy example (3 vectors by hand).	[ ]
10	Finish the Vector Databases course. Note exact search vs approximate search (ANN).	[ ]
11	Start Hugging Face LLM course, chapters 1-2 (what transformers do).	[ ]
12	Microsoft Generative AI for Beginners: lessons on prompt engineering and RAG.	[ ]
13	Read the pgvector README fully. List its data types, operators and index types.	[ ]
14	REVIEW: Draw a diagram of how RAG flows (documents, chunks, embeddings, database, LLM). Quiz yourself without notes.	[ ]

Week 3: Python essentials (Days 15-21)

Goal: Be able to write small scripts that talk to Postgres and APIs.

Day	Task (about 1.5 to 2 hours)	Done
15	Python for Everybody: variables, strings, conditions. Write 5 small exercises.	[ ]
16	Loops, lists, dictionaries. Write a script that counts words in a text file.	[ ]
17	Functions and reading/writing files. Refactor yesterday's script into functions.	[ ]
18	Work with JSON and the requests library. Call a free public API and print the result.	[ ]
19	Install psycopg. Connect to Postgres in Docker; create a table; INSERT and SELECT from Python.	[ ]
20	pandas basics: read a CSV, clean it, and load it into Postgres.	[ ]
21	REVIEW: Mini script: load a CSV into Postgres, then run 3 analysis queries. Commit to GitHub.	[ ]

Week 4: Embeddings in Python (Days 22-28)

Goal: Generate embeddings yourself and prepare your dataset.

Day	Task (about 1.5 to 2 hours)	Done
22	Install sentence-transformers. Embed 20 sentences with all-MiniLM-L6-v2 and print vector size (384).	[ ]
23	Compute cosine similarity between sentences in Python. See which are nearest.	[ ]
24	Call Ollama from Python (HTTP API). Send a prompt and print the answer.	[ ]
25	Pick your dataset: your Sphinx blog pages or a public text dataset. Download it and clean it.	[ ]
26	Split the text into chunks of 300-500 tokens with overlap. Write a reusable chunk function.	[ ]
27	Embed 1,000 chunks and save them to a file. Measure how long it takes.	[ ]
28	CHECKPOINT: Explain embeddings, vector search and RAG out loud in 5 minutes. Fix any weak spots.	[ ]

Week 5: pgvector basics (Days 29-35)

Goal: Store embeddings in Postgres and run similarity queries.

Day	Task (about 1.5 to 2 hours)	Done
29	Lab 1: Run Postgres 16/17 with pgvector in Docker. Run CREATE EXTENSION vector. Check version.	[ ]
30	Lab 2: Create a table with a vector(384) column plus text and metadata columns.	[ ]
31	Lab 2 (cont.): Insert about 10k embedded rows from your dataset using psycopg batch inserts.	[ ]
32	Lab 3: Run similarity queries with the operators for L2, cosine and inner product. Compare top 5 results.	[ ]
33	Run EXPLAIN ANALYZE on each query. Note it uses a sequential scan without an index.	[ ]
34	Add WHERE filters on metadata columns with vector search. Test results and timing.	[ ]
35	REVIEW: Write blog post 1 draft: Getting started with pgvector for DBAs.	[ ]

Week 6: Indexing and tuning (Days 36-42)

Goal: Compare IVFFlat and HNSW and measure recall.

Day	Task (about 1.5 to 2 hours)	Done
36	Scale the table to 100k-500k rows. Record baseline query latency with no index.	[ ]
37	Lab 4: Build an IVFFlat index. Record build time and index size.	[ ]
38	Tune IVFFlat: change lists and probes. Record latency for each setting.	[ ]
39	Lab 5: Build an HNSW index. Record build time, size and maintenance_work_mem used.	[ ]
40	Tune HNSW: change m, ef_construction and hnsw.ef_search. Record latency.	[ ]
41	Lab 6: Measure recall by comparing index results with exact results. Build a speed vs recall table.	[ ]
42	REVIEW: Publish blog post 2: pgvector indexes compared (IVFFlat vs HNSW).	[ ]

Week 7: Build the RAG project (Days 43-49)

Goal: Working Ask-my-docs system on PostgreSQL.

Day	Task (about 1.5 to 2 hours)	Done
43	Lab 7: Ingestion script: read PDFs or markdown files from your docs folder.	[ ]
44	Chunk, embed and store documents with metadata (source, page, created date).	[ ]
45	Test retrieval quality: ask 10 questions and check whether the right chunks come back.	[ ]
46	Lab 8: Add full-text search (tsvector with a GIN index) to the same table.	[ ]
47	Combine full-text rank and vector similarity into a hybrid search query.	[ ]

Day	Task (about 1.5 to 2 hours)	Done
48	Lab 9: Pass the top 5 chunks to a local Ollama model as context and generate answers.	[ ]
49	REVIEW: Publish blog post 3: Running RAG on PostgreSQL. Push all code and README to GitHub.	[ ]

Week 8: Production-style DBA work (Days 50-56)

Goal: Test how the vector workload behaves under real DBA conditions.

Day	Task (about 1.5 to 2 hours)	Done
50	Lab 10: Load test with pgbench custom scripts (or Locust) running vector queries.	[ ]
51	Watch pg_stat_statements, cache hit ratio and memory while load testing. Note bottlenecks.	[ ]
52	Lab 11: Run heavy updates and deletes on the vector table. Check bloat and vacuum behavior.	[ ]
53	Take pg_dump and pg_basebackup backups. Record backup size and restore time.	[ ]
54	Lab 12: Set up a streaming replica. Confirm vector queries work on the standby.	[ ]
55	Try partitioning the table, and read about pgvectorscale for larger datasets.	[ ]
56	REVIEW: Write a runbook: Operating pgvector in production (sizing, tuning, backup, monitoring).	[ ]

Week 9: Production depth (Days 57-63)

Goal: Handle the problems you will face with real vector databases.

Day	Task (about 1.5 to 2 hours)	Done
57	Index build tuning: maintenance_work_mem, parallel maintenance workers, build time vs size.	[ ]
58	Reduce storage with halfvec or quantization options. Measure recall impact.	[ ]
59	Security: roles, row-level security for multi-tenant embeddings.	[ ]
60	Monitoring: postgres_exporter with Prometheus and Grafana. Add latency and index size panels.	[ ]
61	High availability: read about Patroni and replica routing for vector workloads.	[ ]
62	Upgrades and re-embedding strategy: what happens when you change the embedding model.	[ ]
63	REVIEW: Update your runbook with these topics. Test one failure scenario (replica failover).	[ ]

Week 10: Next-level skills and career (Days 64-70)

Goal: Turn learning into job-ready proof.

Day	Task (about 1.5 to 2 hours)	Done
64	MLOps basics: read about MLflow and how models are versioned and deployed.	[ ]
65	Try an LLM-assisted query tuning workflow: paste EXPLAIN output into an LLM and verify its advice.	[ ]
66	Explore managed vector options (Neon, Supabase, cloud Postgres) and compare with self-hosted.	[ ]
67	Polish the GitHub README: architecture diagram, benchmark tables, how to run.	[ ]
68	Update resume and LinkedIn: add pgvector, RAG, vector index tuning, and your blog links.	[ ]
69	Share your blog posts and repo on LinkedIn; start applying to roles that mention pgvector or AI data platforms.	[ ]
70	REVIEW: Plan the next 10 weeks: pick one of MLOps, LLM observability, or advanced Postgres internals.	[ ]